

Lección 17: Sentry Alertas y Playbooks

Respuesta Automática a Incidents

Agenda

- ¿Por qué Alertas Inteligentes?
 - Tipos de Alertas
 - Configuración de Thresholds
 - Playbooks para Respuesta
 - Alert Fatigue Prevention
 - Integración con Teams
-

El Problema de las Alertas

Sin alertas inteligentes:

Escenario típico:

1. Error ocurre en producción
2. Nadie se da cuenta por horas
3. Usuarios reportan el problema
4. Equipo descubre en pánico
5. Fix urgente sin contexto

Resultado: Tiempo de Detección = horas
 Tiempo de Respuesta = más horas

Con alertas inteligentes:

1. Error ocurre
2. Alerta inmediata al equipo
3. Contexto completo disponible
4. Fix informado en minutos

Resultado: Tiempo de Detección = segundos
Tiempo de Respuesta = minutos

Tipos de Alertas (1/2)

1. Alertas de Tasa de Error:

Activador: Tasa de error > umbral
Ejemplo: "Error rate 5% (normal: 0.5%)"

Cuándo alertar:

- Tasa de error aumenta 200%+
- Tasa de error > 2% absoluto
- Aparecen nuevos tipos de errores

2. Alertas de Salud de Release:

Activador: Nuevo release degrada métricas
Ejemplo: "v2.1.0 has 3x more errors than v2.0.9"

Cuándo alertar:

- Tasa de crash aumenta
 - Nuevos patrones de error
 - Impacto de usuario > umbral
-

Tipos de Alertas (2/2)

3. Alertas de Rendimiento:

Activador: El rendimiento se degrada
Ejemplo: "LCP increased from 1.5s to 4.2s"

Cuándo alertar:

- Core Web Vitals se degradan
- Tiempo de respuesta > presupuesto
- Tasa de timeout aumenta

4. Alertas de Impacto de Usuario:

Activador: Muchos usuarios afectados

Ejemplo: "1,000+ users experiencing errors"

Cuándo alertar:

- Usuarios únicos afectados > umbral
- Segmentos críticos de usuarios afectados
- Errores que impactan ingresos

Configuración de Umbrales

Umbral de Tasa de Error:

```
{
  metric: 'error_rate',
  warning: '1%',    // Notificación por email
  critical: '2%',   // PagerDuty/Slack inmediato
  comparison: 'previous_period',
  timeWindow: '5 minutes'
}
```

Umbral de Rendimiento:

```
{
  metric: 'lcp',
  warning: '3000ms', // Aproximándose al presupuesto
  critical: '4000ms', // Presupuesto excedido
  percentile: 'p95', // Percentil 95
  comparison: 'baseline'
}
```

Prevención de Fatiga de Alertas

El problema:

- ✗ Demasiadas alertas → Equipo las ignora
- ✗ Alertas ruidosas → Falsos positivos
- ✗ Sin priorización → Todo parece urgente

Solución:

- ✓ Agrupación de alertas: Agrupa errores similares
 - ✓ Umbrales inteligentes: Aprende de histórico
 - ✓ Niveles de prioridad: Critical > Warning > Info
 - ✓ Reglas de silencio: Ignora conocidos temporalmente
-

Agrupación de Alertas

Sin agrupación:

```
Alert 1: TypeError in user-profile (user 123)
Alert 2: TypeError in user-profile (user 456)
Alert 3: TypeError in user-profile (user 789)
...
Alert 100: TypeError in user-profile (user X)
```

→ 100 notificaciones del mismo problema

Con agrupación:

```
Alert: TypeError in user-profile
Affected: 100 users
Status: CRITICAL
Action: Fix immediately
```

→ 1 notificación con contexto completo

Niveles de Prioridad

CRÍTICO (Respuesta inmediata):

- Errores que impactan ingresos
- Funcionalidad rota visible para usuarios
- Pérdida/corrupción de datos
- Incidentes de seguridad

ADVERTENCIA (Monitorear, planificar fix):

- Degradación de rendimiento o característica no crítica rota
- Tasa de error aumentada (manejeable)

INFORMACIÓN (Seguimiento, sin urgencia):

- Problemas de servicios de terceros
- Errores esperados (input inválido)
- Bugs de bajo impacto

Playbooks: Respuesta Estructurada

¿Qué es un Playbook?

Un playbook es una guía paso a paso para responder a un tipo específico de incidente.

Beneficios:

- ✓ Respuesta rápida y consistente
- ✓ Menos pánico en incidentes
- ✓ Onboarding de nuevo team
- ✓ Post-mortem más fácil

Playbook: Tasa de Error Alta

Playbook: Tasa de Error Alta

Activador

Tasa de error > 2% por 5+ minutos

Acciones Inmediatas

1. Revisar dashboard de Sentry para detalles del error
2. Identificar versión de release afectada
3. Verificar si un segmento específico de usuarios está afectado
4. Verificar estado de infraestructura (AWS, CDN)

Árbol de Decisión

- ¿Nuevo release? → Considerar rollback
- ¿Problema de infraestructura? → Escalar/reiniciar
- ¿Problema de terceros? → Habilitar fallback

Comunicación

- Publicar en canal #incidents
- Actualizar página de estado si afecta usuarios
- Notificar stakeholders si es crítico

Resolución

- Desplegar fix o hacer rollback
- Monitorear tasa de error por 15 min
- Marcar incidente como resuelto
- Programar post-mortem

Playbook: Degradación de Rendimiento

Playbook: Degradación de Rendimiento

Activador

LCP > 4s para usuarios p95 por 10+ minutos

Acciones Inmediatas

1. Revisar dashboard de Performance de Sentry
2. Identificar transacciones lentas
3. Revisar cambios recientes de código
4. Verificar latencia de servicios externos

Investigación

- ¿Queries de base de datos lentas?
- ¿Respuestas de API demoradas?
- ¿Bundle grande desplegado?
- ¿Problemas de CDN?

Mitigación

- Habilitar optimizaciones de rendimiento
- Escalar infraestructura
- Cache agresivo
- Rollback si es necesario

Comunicación

- Actualización transparente a usuarios si es severo
- Publicar progreso en canal #performance

Integración con Equipos

Integración con Slack:

```
// Notificación automática a Slack
{
  channel: '#alerts',
  message: '🚨 CRITICAL: High error rate',
  details: {
    error_rate: '3.5%',
    affected_users: 1234,
    release: 'v2.1.0'
  },
  actions: [
    'View in Sentry',
    'Start incident',
    'Rollback release'
  ]
}
```

Reglas de Silencio

Cuándo silenciar:

- ✓ Problema conocido de terceros (temporal)
- ✓ Errores esperados durante migración
- ✓ Bugs de baja prioridad programados para fix
- ✓ Errores en características deprecadas

Ejemplo:

```
{  
  rule: 'Silence NetworkError',  
  reason: 'Known CDN issue, monitoring',  
  duration: '2 hours',  
  conditions: {  
    error_type: 'NetworkError',  
    message_contains: 'cdn.example.com'  
  }  
}
```

Métricas de Alertas

Métricas de Efectividad:

Tiempo de Detección (TTD):

< 5 min = Excelente ✓

5-15 min = Bueno ⚠

> 15 min = Necesita mejora ✗

Tiempo de Respuesta (TTR):

< 15 min = Excelente ✓

15-60 min = Bueno ⚠

> 60 min = Necesita mejora ✗

Precisión de Alertas:

> 90% accionable = Bueno ✓

70-90% accionable = Necesita ajuste ⚠

< 70% accionable = Demasiado ruidoso ✗

Proceso de Post-Mortem

Después de cada incidente:

1. Línea de tiempo: ¿Qué pasó y cuándo?
2. Causa raíz: ¿Por qué sucedió?
3. Impacto: ¿A quiénes afectó?
4. Respuesta: ¿Qué hicimos bien/mal?
5. Acciones: ¿Cómo prevenimos esto?

NO cultura de culpa

SÍ cultura de aprendizaje

Ejercicio 1: Configurar Alert Rule Inteligente

Prompt:

Actúa como un SRE configurando alertas inteligentes para prevenir incidents

CONTEXT0: Alert Fatigue = demasiadas alertas → equipo ignora todas. Sin ale problemas descubiertos por usuarios (malo). Alertas inteligentes: threshold basado en datos reales (error rate > 1% = anormal). Time window: 5 minutos (evita falsos positivos de spikes momentáneos). MTTR (Mean Time To Recovery objetivo < 15 min (alertas automáticas críticas). Sentry Alert Rules: condi + acciones (email, Slack, PagerDuty). Environment filters: production (NO a en dev). Proactive monitoring: detectar problemas ANTES que usuarios report

TAREA: Configura alert rule en Sentry para detectar error rate alto en cart

CONFIGURACIÓN EN SENTRY DASHBOARD:

1. Ir a sentry.io → Alerts → Create Alert Rule
2. Select project: [tu proyecto]

ALERT RULE CONFIGURATION:

- Name: "High Error Rate - Cart"
- Condition: "When error rate > 1% in 5 minutes"
- Environment filter: production (excluir development)
- Tags filter (opcional): component:cart (si usas tags)

ACTIONS (Notificaciones):

- Send email to: [tu email]
- Send Slack notification to: #alerts (si tienes Slack integrado)

– Opcional: PagerDuty para on-call rotation

TESTING ALERT:

Generar errores artificiales rápidamente:

// En browser console o `test` button

```
Array.from({length: 15}).forEach(() => {  
  Sentry.captureException(new Error('Test alert - high error rate'))  
})
```

VALIDACIÓN:

1. Errores generados → error rate spike > 1%
2. Esperar ~2-3 minutos (Sentry procesa)
3. Verificar:
 - Email recibido: `"High Error Rate - Cart (2.3%)"`
 - Slack notification (si configurado)
 - Alert tiene link a Sentry issues

VALIDACIÓN: Alert notification debe llegar con error rate % y link a dashbo

Aprende: Alertas basadas en thresholds detectan anomalías

antes que usuarios las experimenten

Ejercicio 2: Documentar Playbook de Respuesta

Prompt:

Actúa como un incident manager documentando playbook de respuesta para prod

CONTEXT0: Playbook = checklist documentado de pasos para resolver incident específico. Sin playbook: pánico → investigación desde cero → tiempo perdid MTTR alto. Con playbook: pasos claros → diagnóstico rápido → fix sistemátic MTTR bajo. Runbook vs Playbook: runbook = procedimiento operacional, playbo respuesta a incident. Incident Response: 1) Detect, 2) Diagnose, 3) Fix, 4) Document. MTTR target: < 15 min para incidents críticos. Escalation path: cuándo escalar a on-call engineer. Documentation as Code: playbook en código (NO wiki externo que nadie lee).

TAREA: Documenta playbook para cart errors como comentario JSDoc en código.

UBICACIÓN:

- Archivo: `src/context/CartContext.tsx` (o archivo principal de `cart`)
- Formato: JSDoc comment `/** */` al inicio del archivo

PLAYBOOK STRUCTURE:

```
/**
 * PLAYBOOK: Cart Error Response
 *
 * TRIGGER: Error rate > 2% in cart operations
 *
 * DIAGNOSIS STEPS:
 * 1. Check product API status: /api/health
 * 2. Verify cart data structure in latest Sentry issues
 * 3. Check recent deployments (last 2 hours) in CI/CD
 * 4. Review Sentry breadcrumbs for user journey pattern
 *
 * REMEDIATION:
 * - If API down: Enable fallback mode (static product data)
 * - If data structure issue: Rollback to previous release
 * - If user input issue: Add validation + hotfix deploy
 *
 * INCIDENT MANAGEMENT:
 * - Create incident in Sentry: Mark as "Critical"
 * - Notify team: Post in #incidents Slack channel
 * - Update status page: "Cart experiencing issues"
 *
 * TARGETS:
 * - MTTR: < 15 minutes
 * - Escalation: After 30 min without resolution → call on-call
 * - Post-mortem: Required for all > 5 min downtime
 *
 * CONTACT: oncall-engineer@example.com
 */
```

ELEMENTOS REQUERIDOS:

- TRIGGER: Qué condición activa el playbook
- DIAGNOSIS: Pasos de investigación ordenados
- REMEDIATION: Acciones de fix por tipo de problema
- INCIDENT MANAGEMENT: Comunicación y tracking
- TARGETS: MTTR y escalation timing

VALIDACIÓN: Playbook debe estar como comentario en archivo de producción

Aprende: Playbooks documentados en código reducen MTTR

y pánico durante incidents

Puntos Clave

1. **Alertas Inteligentes:** Detección en segundos, no horas
2. **Tipos:** Error rate, release health, performance, user impact
3. **Thresholds:** Configurados por severidad
4. **Prevent Fatigue:** Grouping, priorización, silence rules
5. **Playbooks:** Respuesta estructurada y consistente
6. **Integration:** Slack, PagerDuty, equipos
7. **Post-Mortems:** Aprende de cada incident